RIT ALGO2e: Strategy Report

Main strategy: `algo2e_fin.py` (trading code in Appendix A)

(a) What was your strategy going in?

The problem. ALGO2e is a liquidity-provision case: three stocks (CNR ≈\$160, RY ≈\$100, AC ≈\$25), a 25,000-share gross and net limit with fines for breaches, and a fee schedule that pays a rebate for resting orders on two of the three stocks. Market making is the natural way to earn money here: quote a bid and an ask, collect the spread and the rebate, and keep inventory near zero. It is also the natural way to lose money, because in a trending market a market maker keeps buying a falling price (or selling a rising one) and ends up holding a large position against the move. We did not know in advance which kind of market the case would produce. Our working guess was that prices would mostly mean-revert around an anchor, with the possibility of trends or shocks injected during the case. The strategy therefore had to make money in a calm market, survive a trending one, and switch between the two without anyone touching it.

The design: a regime-switching system with three parts.

1. **Market maker (base income in calm markets).** Passive quotes on every stock, sized according to how volatile the stock currently is, with an inventory brake that makes it progressively harder to add to a position and easier to reduce it.
2. **Trend engine (insurance against our guess being wrong).** When a stock starts moving steadily in one direction, quoting on that stock stops and the engine takes a large position *with* the move, held with a trailing stop that is scaled to the stock's volatility.
3. **Risk layer (overrides both).** Only limit orders are used, every order is checked against worst-case exposure before it is sent, and an account-level circuit breaker and end-of-case unwind sit on top. Our explicit objective was profit with *zero fines*.

Each stock is handled every pass of the loop (up to 40 times a second) in a fixed order of precedence: end-of-case unwind or circuit breaker first, then the trend engine, then market-making loss controls, and only then new quotes. The first rule that applies wins, so safety always takes priority over making money.

How the bot knows which regime it is in. For every stock the bot keeps a few simple, robust measurements: the *EMA gap*, the difference between a fast (1 s) and a slow (8 s) exponential moving average of the mid-price, measured in ticks, which says how far the price has run from its recent past; the *Kaufman efficiency ratio* over 10 s, |net move|/total path, which is close to 1 when the price moves in a straight line and close to 0 when it goes back and forth; the width of its *Bollinger bands* relative to their own 2-minute baseline, which classifies the stock as calm, normal or stressed; a short-term *volatility* estimate; and a *t-statistic* of the drift over 20 case ticks for slow, grinding trends. A stock is labelled TREND when the efficiency ratio is at least 0.45 and the move is at least 12 ticks, and it keeps that label until the ratio falls below 0.33. This hysteresis stops the label from flickering on the boundary. Importantly, the label alone never triggers a trade. The trend engine also requires the EMA gap to exceed $\max(12 \text{ ticks}, 5\sigma)$, the spread to be at most 6 ticks, and the signal to persist for 0.8 s. Because the thresholds are expressed in units of the stock's own volatility, the same rules work for a \$25 stock and a \$160 stock, and a noisy stock needs a bigger move before the engine believes it.

What we decided against. We also built a directional mean-reversion tilt: instead of steering inventory to zero, it targets a long position when the price is low in its Bollinger band and a short one when it is high (up to 8,000–12,000 shares). In testing it did worse than plain inventory-to-zero market making, because it is holding inventory against the move at exactly the moment a breakout starts. It was switched off for the graded run (`MR_MAX_INV = 0`). Several other rules came from our practice sessions: market orders into a thin AC book lost tens of thousands of dollars in a second, so every aggressive order is a limit order capped within 2% of fair value; and a bot that reads its own resting quotes as "the market" keeps outbidding itself, so the bot strips its own orders from the book before computing any price.

(b) What changes did you make to handle multiple securities and fees/rebates?

Nothing about the securities is hard-coded. At start-up the bot reads `/securities` for the list of tradeable tickers and each ticker's tick size, taker fee and maker rebate, and `/limits` for the gross and net limits and the fine schedule. Its own caps are derived from these values, so the same script runs unchanged on two, three or ten securities and on a different limit.

Each stock thinks for itself; all stocks share one risk budget. Every stock has its own signals, regime label, mode (market making, trend, or exiting a trend), stops, loss pauses and a learned entry threshold that rises after losing

trend trades on that stock. What the stocks cannot each have is their own limit, because the 25,000-share limit is shared. We handled this in four ways:

- **Aggregate worst-case check.** Before any order is sent, the bot assumes that every resting order, every order still in flight and the new order all fill at once, summed across all stocks, and requires gross and net exposure to stay below 93% of the limit (23,250 shares). If the order does not fit, it is shrunk by 40% at a time and dropped if it still does not fit.
- **Capacity allocation.** The trend engine's maximum size on one stock is 0.95× the cap minus whatever is held in the other stocks. When RY's trend started the other books were almost flat, so RY could take about 22,000 shares; had AC been trending at the same time, RY would have received only what was left.
- **Per-stock inventory caps.** Market-making inventory is limited to 9,000 shares per stock, so one stock cannot absorb the whole budget through passive fills.
- **Priority when room is tight.** Aggressive orders (stops, trend trades, unwind) are processed first, then the quotes of the stock with the healthiest wide spread, which is the most profitable to quote, then the remaining quotes.

Order books for all stocks are fetched in parallel so that adding a stock does not slow the loop. Because RIT confirms orders and updates positions with a delay, the bot keeps its own registry of orders: a just-sent order counts as live before RIT lists it, and an order that has disappeared still counts toward exposure for a second, until the position catches up.

Fees and rebates. The useful way to read the fee schedule is as a fraction of the one-tick ($0.01) spread, because that is all a market maker earns per share:

	Maker rebate (resting)	Taker fee (crossing)	What it means for us
CNR	+$0.0023 (23% of a tick)	$0.0027 (27% of a tick)	rest; crossing costs > ¼ of the spread
AC	+$0.0011 (11%)	$0.0015 (15%)	rest
RY	−$0.0020 (resting *pays* 20%)	−$0.0014 (crossing *earns* 14%)	resting costs; crossing is cheap

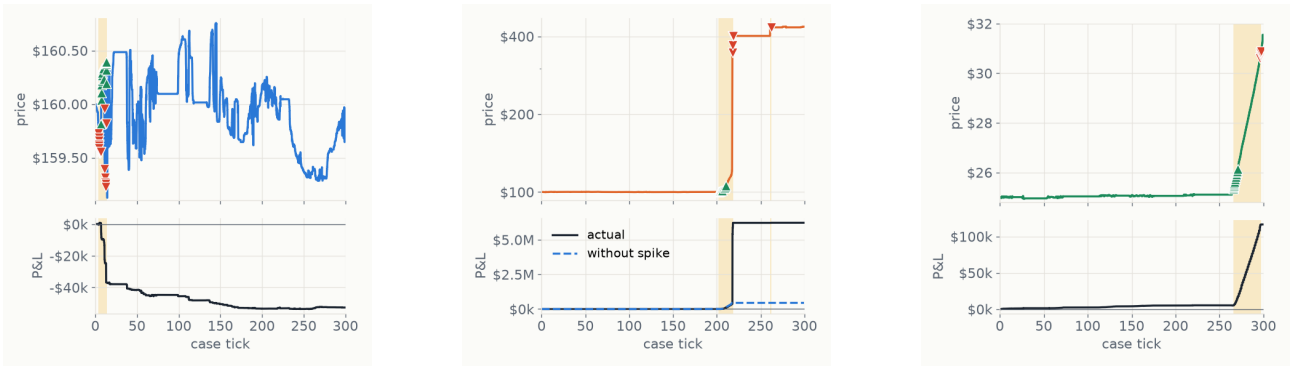
A 1,000-share round trip on CNR earns $10 of spread plus $4.60 of rebates if both sides rest, but crossing both sides would cost the $10 spread plus $5.40 in fees. That asymmetry drove three choices. First, **passive-first execution**: market making only rests orders, and we cross the spread only when we must (stops, trend entries and exits, the final unwind), always with capped limit orders. Second, **size where the economics are best**: full size (5,000 shares a side) is quoted only when the market is calm or the spread is a "healthy" 2–6 ticks, where spread plus rebate can pay for the inventory risk; on a 2-tick spread we step inside only on the side that reduces our position. Third, **RY is different**: resting there costs money, so market making on RY is structurally weak, while our RY trend trades were executed aggressively, which is the cheaper side on that stock. In this version the fee schedule shaped the design rather than entering the quoting formula itself; making quotes explicitly fee-aware, for example not resting on RY at all, is the first change we would make.

(c) What happened during the simulation? What was your size strategy?

The market we got. For the first 200 case ticks all three prices behaved like mean-reverting series. Their 10-tick variance ratios were 0.19–0.26 (a random walk scores 1), and consecutive returns were negatively correlated (−0.21 to −0.33), so our working assumption held for two-thirds of the case. After tick 200 the picture changed: RY broke out, AC trended steadily, and CNR stayed inside its band to the end (Figure 1).

What the bot did, stock by stock.

- **CNR (ticks 3–13): false starts, then the learning rule worked.** The engine took three positions (short, long, short) that reversed almost immediately, losing $36.7k. In the opening seconds CNR's book was thin, with spreads of 11 to 100 ticks and price jumps of 30–100 cents per tick, while the volatility estimate was still at its starting value. Noise therefore looked like a trend. Each loss raised CNR's entry bar by 1.4×, three losses in a row locked the engine out of CNR, and it never traded CNR directionally again. The account-level circuit breaker fired once, at tick 38, flattened everything and paused for eight seconds.
- **RY (ticks 200–218): the system working as designed.** At tick 200 the efficiency ratio jumped from 0 to 1.0, a straight-line move. At tick 201.6 the EMA gap had stayed above 12 ticks for 0.8 s and the trend engine took over. Its first order was about 12,000 shares; by tick 210 it held 22,062 shares (88% of the limit), bought between $100.2 and $105.5, and the wide trailing stop let it ride the drift to $118. At tick 217 the RY book turned one-sided: the asks vanished and the bids jumped to $346 and then $402, following Group 12's activity. The engine sold the whole position into those bids in five orders over about four seconds, at an average of roughly



(a) CNR: −$53k **(b)** RY: +$6.22M **(c)** AC: +$117k

Figure 1: Price (top) and cumulative P&L (bottom) per stock. Triangles: trend-engine buys and sells; shading: trend mode; dashed line (RY): P&L without the tick-217 event.

$383.

- **AC (ticks 265–296): a clean trend trade.** AC began a straight climb from $25.1 to $31.5. The engine entered at the start, held about 22,000 shares and closed in the end-of-case unwind.

Why market making did not make money, even though prices mean-reverted. This was the most instructive result. With a one-tick spread, a successful round trip earns about $0.01 per share plus the rebate. The inventory stop, however, cuts a losing position once it is down $\max(5 \text{ ticks}, 3\sigma)$. With roughly +1 tick per winning trade and −5 ticks per stopped trade, the market maker needs about five wins in every six trades just to break even. In this market prices moved in jumps of several cents rather than one-cent wiggles, so stops fired often (86 times on CNR), and the orders that filled were disproportionately those a moving price ran through (adverse selection). Mean reversion in the *price* is not the same thing as profitable market making; the spread has to be wide relative to the size of the jumps.

Size strategy: be big only when the evidence is strong or the risk is small.

- **Market-making quotes** were 5,000 shares a side in calm markets or healthy 2–6 tick spreads, 3,000 normally, half that when stressed, and 2,000 in thin books, so quote size falls exactly when the risk of being run over rises.
- **The inventory brake** uses $r = |\text{position}|/9{,}000$. The side that would add to the position is multiplied by e^{-3r} and moved away from the market by $\text{round}(4r^{1.5})$ ticks, and it is switched off entirely at $r \geq 0.85$ (7,650 shares), while the reducing side is enlarged by $(1 + r)$. For example, holding 4,500 shares ($r = 0.5$) cuts a 3,000-share adding quote to about 700 shares placed one tick further away, and raises the reducing quote to 4,500. This is a discrete version of the inventory skew in the Avellaneda–Stoikov market-making model.
- **Trend sizing** starts with 60% of the available capacity and grows with the strength of the signal up to 95% of the cap (about 22,000 shares), in orders of at most 5,000. The trend engine is the only part allowed to use most of the limit, because it only acts when the evidence is strong, and it carries a trailing stop.
- **Limits and unwinding.** On top of the 93% pre-trade check, a safety net cuts the largest position immediately if the actual position ever passes 96% of the limit; the $40k drawdown breaker flattens the book; and about 20 s before the close the bot stops adding risk and about 8 s before it crosses to finish flat. After any trend exit the stock sells down in an "exit" mode with capped limit orders. Over the whole run the peak gross position was 23,088 and the peak net position 22,837, against a limit of 25,000.

Our Python script and how to use it. The trading code is listed in the Appendix (`algo2e_strategy.py`). It needs Python 3 and the `requests` package. Start the RIT client with the API enabled, note the API key and port, and run:

```
python algo2e_strategy.py <API_KEY> <PORT> [CASE_MINUTES] # e.g. python algo2e_strategy.py abcde 15000 20
```

If started before the case begins, it waits until the case status is `ACTIVE`; without `CASE_MINUTES` it learns the length of a tick from the case clock and scales its time windows accordingly. Every parameter sits in one configuration block at the top of the file. Ctrl-C cancels all orders and flattens the book. While running it prints every fill, mode change and stop, and once a second a status line per stock with its position, quotes and regime. It writes two logs: `logs/v3_ticks_*.csv`, one row per loop with prices, signals, positions, modes and P&L for every stock, and `logs/v3_events_*.csv`, every order, cancel, fill and decision with its reason. All numbers in this report come from these logs. `dir.py` (trend engine only) and `mm.py` (market making only) are the same code with one engine disabled.

Testing before the case. During development we tested the bot against a fake exchange that reproduces RIT's order latency, position-reporting lag, rebates and fees, with eleven market scenarios. Re-running our development script through all of them (90 s each) gave: calm, mean-reverting and slow-grind markets between −$932 and +$440; trending, reversing and crashing markets +$41k to +$124k; spikes, sweeps and one-sided order flow +$12k to +$49k; and a

whipsaw market (short trends that reverse) −$44k. Every run ended flat with peak gross at most 23,200 shares. The live result has the same shape: break-even in calm markets, profits in trends, and the whipsaw case is the same failure mode as CNR's opening.

A note on the following round. In the next round our RIT client crashed, so we had cancelled the script and only started it after the round was under way (it first joined at tick 49 of 300). It could not run the strategy as designed, and the late session ended with a small loss of $3,427. We mention it for completeness only.

(d) What was your P&L?

The main strategy finished at $6,288,771. This is the server's net liquidation value. It equals the sum of the three stocks' P&L to the cent, which is what we would expect if no fines were charged. The two variants ran on separate accounts in the same market and are reported separately:

	CNR	RY	AC	Total
`algo2e_fin.py` (market making + trend; main)	−$52,646	$6,224,827	$116,590	**$6,288,771**
`dir.py` (trend engine only)	−$77,895	$6,238,173	$118,130	$6,278,408
Market making only	−$28,719	−$6,147	$6,434	−$28,433

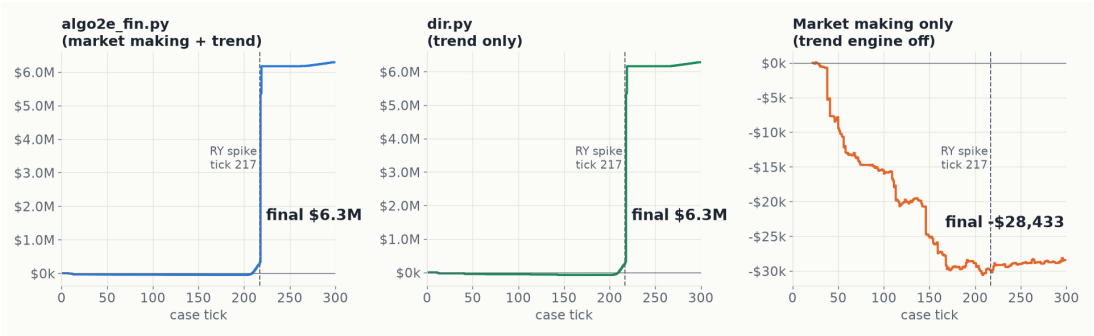


Figure 2: Total P&L through the case for the three builds; dashed line: the RY event at tick 217.

Where the money came from. Splitting the main strategy's P&L by the mode the bot was in shows a clear division of labour. The trend engine was in control for less than 7% of the time on any stock (about 2% on CNR, 4% on RY and 6% on AC) and earned +$6.30M net: +$5.78M from selling RY into the dislocation, +$451k from riding RY's drift before it, +$112k on AC, less $36k on CNR's early false starts. Market making, active the rest of the time, finished at about −$15k. Running three builds side by side in the same market makes this a controlled comparison: the same trend engine produced almost the same result with or without market making, while market making alone lost money. `dir.py`'s faster entries (8-tick instead of 12-tick threshold) lost more on CNR, which illustrates the trade-off between catching a move early and reacting to noise.

With and without the RY event. About 92% of the profit came from one event, so we also measured what the strategy earned on its own terms. Re-valuing the RY exit at the last normal bid before the book broke ($121.08), with everything else unchanged, gives about **$511k**, and between $244k and $559k for exit prices from $109 to $123. This still includes the RY drift that preceded the event, so it is the optimistic reading; ticks 0–199 on their own were a loss of about $52k.

What the mean-reversion tilt would have added. Replaying the logged Bollinger z-scores and regime labels, and holding the position the tilt would have targeted, gives about +$37k at 8,000 shares and +$56k at 12,000 before trading costs, and roughly +$5k to +$20k after them. Almost all of it falls before tick 200, because the tilt switches itself off when a stock is in TREND, so it would not have been short RY into the event. At under 1% of the total, this supports our decision to leave it off.

How we read the result. We did not predict the RY event, and we do not claim to have. What the system did was put the right position on by rule before the event: regime detection moved RY into trend mode within about one case tick of the drift starting, the trend engine sized up to 88% of the limit, and the volatility-scaled trailing stop kept the position on until the book broke. The position stayed within limits for the whole case and no fines were charged. The run also showed the two changes we would make first: a warm-up period before the trend engine is allowed to trade, and fee-aware quoting restricted to stocks where resting orders earn a rebate.

A Python code: `algo2e_strategy.py`

This is the trading code of our main strategy. Development-only code (the offline test exchange and experiment hooks) and two features that did not trade in the graded run are omitted. Run it with `python algo2e_strategy.py <API_KEY> <PORT> [CASE_MINUTES]`.

```python
#!/usr/bin/env python3
"""RIT ALGO2e strategy: market making + trend engine + risk layer.

Usage: python algo2e_strategy.py <API_KEY> <PORT> [CASE_MINUTES]
Logs: logs/v3_ticks_<time>.csv and logs/v3_events_<time>.csv
"""
import argparse
import collections
import csv
import math
import os
import signal
import threading
import time
from concurrent.futures import ThreadPoolExecutor
from datetime import datetime

import requests

# ---------------------------------------------------------------------------- config
FEES = {"CNR": (0.0027, 0.0023), "RY": (-0.0014, -0.0020), "AC": (0.0015, 0.0011)}
DEFAULT_TICKERS = ["CNR", "RY", "AC"]

MAX_ORDER = 5000
DEFAULT_LIMIT = 25000
CAP_FRACTION = 0.93
SAFETY_FRACTION = 0.96

LOOP_MIN_DT = 0.025
SLOW_POLL_SEC = 0.5
HTTP_TIMEOUT = 1.5

QUOTE_SIZE = 3000
QUOTE_SIZE_CALM = 5000
WIDE_OK_MIN, WIDE_OK_MAX = 2, 6
EXP_K = 3.0
HALT_FRAC = 0.85
FADE_MAX_TICKS = 4
MM_INV_CAP = 9000
LEAN_MAX_INV = 3000
IMPROVE_MIN_SPREAD = 2
KEEP_FRAC = 0.35
WIDE_SPREAD_TICKS = 12
WIDE_EDGE_TICKS = 4
WIDE_SIZE = 2000

BB_WINDOW_SEC = 30.0
BB_LONG_TAU = 120.0
CALM_RATIO = 1.3
STRESS_RATIO = 2.0
CALM_MAX_STD_TICKS = 4.0

STOP_MIN_TICKS = 5
STOP_VOL_K = 3.0
STOP_COOLDOWN = 3.0

FAST_TAU = 1.0
SLOW_TAU = 8.0
FAIR_TAU = 3.0
DIR_MIN_TICKS = 12
DIR_VOL_K = 5.0
DIR_CONFIRM_SEC = 0.8
DIR_MAX_SPREAD_TICKS = 6
DIR_FRACTION = 0.95
DIR_EXIT_FRAC = 0.25
DIR_TRAIL_TICKS = 15
DIR_TRAIL_VOL_K = 6.0
DIR_COOLDOWN = 4.0
DIR_STALL_SEC = 8.0
DIR_MIN_SIZE = 2000

ER_WINDOW_SEC = 10.0
ER_TREND = 0.45
ER_CHOP = 0.25
DIR_MULT_MAX = 3.0
CHOP_LOCKOUT_RIDES = 3
CHOP_LOCKOUT_SEC = 60.0
TOXIC_STOPS = 3
TOXIC_WINDOW = 60.0
TOXIC_SEC = 30.0
MM_LOSS_WINDOW = 45.0
```

```
MM_LOSS_LIMIT = 2000.0
MM_PAUSE_SEC = [20.0, 45.0, 90.0, 180.0]

VISIBILITY_GRACE = 1.5
GONE_HOLD = 1.0
CANCEL_RESEND = 1.0
AGG_INTERVAL = 1.0
AGG_TTL = 0.6

SLOW_TICKS = 20
SLOW_T_ENTER = 3.0
SLOW_T_LEAN = 2.0
SLOW_T_EXIT = 0.5
SLOW_MIN_TICKS = 15
SLOW_FRACTION = 0.5
SLOW_STALL_TICKS = 10

AGG_THROUGH_TICKS = 3
AGG_MAX_DEV = 0.02
AGG_CHUNK = 5000
LATENCY_SEC = 0.05
SPEED_PULL_TICKS = 3.0

BREAKER_DD = 40000.0
BREAKER_PAUSE = 8.0

UNWIND_START_SEC = 20.0
UNWIND_HARD_SEC = 8.0

HEARTBEAT_SEC = 1.0
LOG_DIR = "logs"

SPT_REF = 2.0
_SCALED = ["ER_WINDOW_SEC", "BB_WINDOW_SEC", "BB_LONG_TAU", "SLOW_TAU", "FAIR_TAU",
           "DIR_STALL_SEC", "MM_LOSS_WINDOW", "TOXIC_WINDOW"]
_BASE = {}


def apply_time_scale(spt):
    g = globals()
    if not _BASE:
        _BASE.update({k: g[k] for k in _SCALED})
    k = max(0.6, min(1.8, math.sqrt(max(spt, 0.05) / SPT_REF)))
    for name in _SCALED:
        g[name] = _BASE[name] * k
    return k


# ---------------------------------------------------------------------------- RIT REST client
class RIT:
    def __init__(self, key, base):
        self.key, self.base = key, base.rstrip("/")
        self.local = threading.local()
        self.lock = threading.Lock()
        self.n429 = 0

    def _sess(self):
        s = getattr(self.local, "s", None)
        if s is None:
            s = requests.Session()
            s.headers.update({"X-API-Key": self.key})
            s.mount("http://", requests.adapters.HTTPAdapter(pool_connections=8, pool_maxsize=32))
            self.local.s = s
        return s

    def req(self, method, path, params=None):
        for attempt in range(4):
            try:
                r = self._sess().request(method, self.base + path, params=params, timeout=HTTP_TIMEOUT)
            except Exception:
                time.sleep(0.05 * (attempt + 1))
                continue
            if r.status_code == 429:
                with self.lock:
                    self.n429 += 1
                wait = 0.05
                try:
                    wait = float(r.json().get("wait", wait))
                except Exception:
                    try:
                        wait = float(r.headers.get("Retry-After", wait))
                    except Exception:
                        pass
                time.sleep(min(max(wait, 0.01), 0.5))
                continue
            if r.status_code == 401:
                return {"_error": 401, "body": r.text[:160]}
            if r.ok:
                try:
```

```
                return r.json()
            except Exception:
                return {}
        return {"_error": r.status_code, "body": r.text[:160]}
    return {"_error": "no-response"}

def case(self): return self.req("GET", "/case")
def trader(self): return self.req("GET", "/trader")
def limits(self): return self.req("GET", "/limits")
def securities(self): return self.req("GET", "/securities")
def book(self, t): return self.req("GET", "/securities/book", {"ticker": t, "limit": 20})
def open_orders(self): return self.req("GET", "/orders", {"status": "OPEN"})

def place(self, t, action, qty, price):
    return self.req("POST", "/orders", {"ticker": t, "type": "LIMIT", "quantity": int(qty),
                                        "action": action, "price": round(price, 4)})

def cancel(self, oid): return self.req("DELETE", f"/orders/{oid}")
def cancel_all(self): return self.req("POST", "/commands/cancel", {"all": 1})


# ---------------------------------------------------------------------- per-security state
class Sec:
    def __init__(self, t):
        self.t = t
        self.tick = 0.01
        self.fee_take, self.fee_make = FEES.get(t, (0.0, 0.0))
        self.pos = 0
        self.vwap = 0.0
        self.realized = self.unrealized = 0.0
        self.bid = self.ask = 0.0
        self.last = 0.0
        self.fast = self.slow = self.fair = None
        self.vol = 1.0
        self.drift = 0.0
        self.vel = 0.0
        self.prev_mid = None
        self.vol_ref_mid = None
        self.vol_ref_t = 0.0
        self.last_t = None
        self.mode = "mm"
        self.dir_side = 0
        self.dir_target = 0
        self.dir_best = 0.0
        self.dir_best_t = 0.0
        self.last_exit = None
        self.trend_since = None
        self.cool_until = 0.0
        self.no_buy_until = 0.0
        self.no_sell_until = 0.0
        self.orders = []
        self.tick_mids = collections.deque(maxlen=SLOW_TICKS + 1)
        self.last_case_tick = None
        self.t_slow = 0.0
        self.net_slow = 0.0
        self.sd_slow = 1.0
        self.dir_kind = "fast"
        self.gap = 0.0
        self.hist = collections.deque()
        self.hist_t = 0.0
        self.er = 0.0
        self.net_ticks = 0.0
        self.regime = "warmup"
        self.dir_mult = 1.0
        self.lose_streak = 0
        self.dir_off_until = 0.0
        self.dir_entry_mid = 0.0
        self.rides = [0, 0]
        self.stop_times = collections.deque()
        self.toxic_until = 0.0
        self.est_pnl = 0.0
        self.est_mid = None
        self.est_pos = 0
        self.pnl_hist = collections.deque()
        self.pnl_hist_t = 0.0
        self.mm_off_until = 0.0
        self.mm_pauses = 0
        self.size_mult = 1.0
        self.size_t = 0.0
        self.bbq = collections.deque()
        self.bb_t = 0.0
        self.bb_z = 0.0
        self.bb_std = 0.0
        self.bw_long = None
        self.vstate = "normal"

    @property
    def spread_ticks(self):
        if self.bid > 0 and self.ask > 0:
```

```
            return round((self.ask - self.bid) / self.tick)
        return 999

    @property
    def mid(self):
        if self.bid > 0 and self.ask > 0:
            return 0.5 * (self.bid + self.ask)
        return self.fair or self.last or 0.0

    def rnd(self, p):
        return round(round(p / self.tick) * self.tick, 6)

    def update_slow(self, case_tick):
        if case_tick is None or case_tick == self.last_case_tick:
            return
        self.last_case_tick = case_tick
        if not (self.bid > 0 and self.ask > self.bid and self.spread_ticks < WIDE_SPREAD_TICKS):
            return
        self.tick_mids.append(self.mid)
        if len(self.tick_mids) >= 12:
            xs = list(self.tick_mids)
            d = [(b - a) / self.tick for a, b in zip(xs, xs[1:])]
            n = len(d)
            mu = sum(d) / n
            sd = math.sqrt(sum((x - mu) ** 2 for x in d) / max(1, n - 1))
            self.sd_slow = max(sd, 0.5)
            self.net_slow = (xs[-1] - xs[0]) / self.tick
            self.t_slow = self.net_slow / (self.sd_slow * math.sqrt(n))

    def update_signals(self, now):
        good = self.bid > 0 and self.ask > self.bid and self.spread_ticks < WIDE_SPREAD_TICKS
        m = self.mid
        if m <= 0:
            return
        if self.fast is None:
            self.fast = self.slow = self.fair = m
            self.vol_ref_mid, self.vol_ref_t, self.last_t = m, now, now
            return
        dt = max(1e-3, now - self.last_t)
        self.last_t = now
        if good:
            if self.est_mid is not None:
                self.est_pnl += self.est_pos * (m - self.est_mid)
            d = self.pos - self.est_pos
            if d:
                self.est_pnl += abs(d) * 0.5 * (self.ask - self.bid) * (1 if self.mode == "mm" else -1)
            self.est_mid, self.est_pos = m, self.pos
            if now - self.pnl_hist_t >= 1.0:
                self.pnl_hist_t = now
                self.pnl_hist.append((now, self.est_pnl))
                while self.pnl_hist and now - self.pnl_hist[0][0] > MM_LOSS_WINDOW:
                    self.pnl_hist.popleft()
            if self.prev_mid is not None:
                inst = (m - self.prev_mid) / self.tick / dt
                self.vel += (1 - math.exp(-dt / 1.0)) * (max(-200.0, min(200.0, inst)) - self.vel)
            self.prev_mid = m
            self.fast += (1 - math.exp(-dt / FAST_TAU)) * (m - self.fast)
            self.slow += (1 - math.exp(-dt / SLOW_TAU)) * (m - self.slow)
            self.fair += (1 - math.exp(-dt / FAIR_TAU)) * (m - self.fair)
            if now - self.vol_ref_t >= 0.5:
                d = max(-50.0, min(50.0, (m - self.vol_ref_mid) / self.tick))
                self.drift = 0.9 * self.drift + 0.1 * d
                self.vol = 0.9 * self.vol + 0.1 * abs(d - self.drift)
                self.vol_ref_mid, self.vol_ref_t = m, now
        self.gap = (self.fast - self.slow) / self.tick
        if good and now - self.bb_t >= 0.25:
            self.bb_t = now
            self.bbq.append((now, m))
            while self.bbq and now - self.bbq[0][0] > BB_WINDOW_SEC:
                self.bbq.popleft()
            if len(self.bbq) >= 20:
                xs = [x[1] for x in self.bbq]
                mu = sum(xs) / len(xs)
                sd = math.sqrt(sum((x - mu) ** 2 for x in xs) / len(xs))
                self.bb_std = max(sd / self.tick, 0.5)
                self.bb_z = (m - mu) / (self.bb_std * self.tick)
                if self.bw_long is None:
                    self.bw_long = self.bb_std
                else:
                    self.bw_long += (1 - math.exp(-0.25 / BB_LONG_TAU)) * (self.bb_std - self.bw_long)
                ratio = self.bb_std / max(self.bw_long, 0.5)
                self.vstate = ("stressed" if ratio > STRESS_RATIO or self.regime == "TREND"
                               else "calm" if (ratio < CALM_RATIO and self.regime == "chop"
                                               and self.bb_std <= CALM_MAX_STD_TICKS) else "normal")
        if good and now - self.hist_t >= 0.25:
            self.hist_t = now
            self.hist.append((now, m))
            while self.hist and now - self.hist[0][0] > ER_WINDOW_SEC:
                self.hist.popleft()
```

```
            if len(self.hist) >= 8 and now - self.hist[0][0] >= 0.6 * ER_WINDOW_SEC:
                mids = [x[1] for x in self.hist]
                path = sum(abs(b - a) for a, b in zip(mids, mids[1:]))
                net = mids[-1] - mids[0]
                self.er = abs(net) / path if path > 0 else 0.0
                self.net_ticks = net / self.tick
                enter = self.er >= ER_TREND and abs(self.net_ticks) >= DIR_MIN_TICKS
                stay = (self.regime == "TREND" and self.er >= ER_TREND - 0.12
                        and abs(self.net_ticks) >= 0.6 * DIR_MIN_TICKS)
                if enter or stay:
                    self.regime = "TREND"
                elif self.er < ER_CHOP:
                    self.regime = "chop"
                else:
                    self.regime = "mixed"

    def dir_threshold(self):
        return max(DIR_MIN_TICKS, DIR_VOL_K * self.vol) * self.dir_mult


# ---------------------------------------------------------------------------- logging
class Log:
    def __init__(self, tickers, tag="v3"):
        os.makedirs(LOG_DIR, exist_ok=True)
        stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        self.tick_path = os.path.join(LOG_DIR, f"{tag}_ticks_{stamp}.csv")
        self.ev_path = os.path.join(LOG_DIR, f"{tag}_events_{stamp}.csv")
        self.tf = open(self.tick_path, "w", newline="")
        self.ef = open(self.ev_path, "w", newline="")
        self.tw = csv.writer(self.tf)
        self.ew = csv.writer(self.ef)
        hdr = ["t", "case_tick", "sec_left", "pnl", "nlv", "gross", "net"]
        for t in tickers:
            hdr += [f"{t}_bid", f"{t}_ask", f"{t}_fair", f"{t}_gap", f"{t}_vol", f"{t}_er", f"{t}_regime",
                    f"{t}_bbz", f"{t}_vstate", f"{t}_tslow", f"{t}_pos", f"{t}_mode", f"{t}_pnl"]
        self.tw.writerow(hdr)
        self.ew.writerow(["t", "case_tick", "event", "ticker", "side", "qty", "price", "pos", "note"])
        self.last_flush = time.time()
        self.lock = threading.Lock()

    def tick_row(self, row):
        with self.lock:
            self.tw.writerow(row)
            self._maybe_flush()

    def event(self, t, ctick, ev, ticker="", side="", qty="", price="", pos="", note="", echo=False):
        with self.lock:
            self.ew.writerow([f"{t:.3f}", ctick, ev, ticker, side, qty,
                              f"{price:.4f}" if isinstance(price, float) else price, pos, note])
            self._maybe_flush()
        if echo:
            px = f" @ {price:.2f}" if isinstance(price, float) and price else ""
            q = f" {qty}" if qty != "" else ""
            print(f"[{t:7.1f}s] {ev.upper():<8} {ticker:<4} {side}{q}{px} {note}", flush=True)

    def _maybe_flush(self):
        if time.time() - self.last_flush > 1.0:
            self.tf.flush()
            self.ef.flush()
            self.last_flush = time.time()

    def close(self):
        with self.lock:
            self.tf.flush(); self.ef.flush()
            self.tf.close(); self.ef.close()


# ---------------------------------------------------------------------------- bot
class Bot:
    def __init__(self, api, minutes=None):
        self.api = api
        self.pool = ThreadPoolExecutor(max_workers=16)
        self.secs = {}
        self.tickers = []
        self.trader_id = None
        self.gross_limit = self.net_limit = DEFAULT_LIMIT
        self.cap_gross = self.cap_net = int(CAP_FRACTION * DEFAULT_LIMIT)
        self.t0 = time.time()
        self.case_tick = 0
        self.tpp = 300
        self.status = "?"
        self.sec_per_tick = None
        self._tick_anchor = None
        self.nlv0 = None
        self.nlv = None
        self.pnl = 0.0
        self.peak = 0.0
        self.paused_until = 0.0
        self.last_slow = 0.0
```

```
        self.last_beat = 0.0
        self.running = True
        self.flattening = False
        self.widest = None
        self.live = {}
        self.lat_est = LATENCY_SEC
        self.over = 0
        self.loops = 0
        self.places = 0
        self.cancels = 0
        self.log = None
        self.unwind_announced = False
        self.hard_announced = False
        self.period = None
        self.minutes = minutes
        self.time_scale = None

    def now(self):
        return time.time() - self.t0

    # ------------------------------------------------------------------------ setup
    def setup(self):
        while True:
            c = self.api.case()
            if isinstance(c, dict) and not c.get("_error"):
                break
            print(f"[setup] cannot reach RIT ({c}); retrying...", flush=True)
            time.sleep(1.0)
        print(f"[case] {c.get('name', '?')} status={c.get('status')} tick={c.get('tick')}/{c.get('ticks_per_period')}")
        self.tpp = int(c.get("ticks_per_period") or self.tpp)
        if self.minutes:
            self.sec_per_tick = self.minutes * 60.0 / self.tpp
            self.time_scale = apply_time_scale(self.sec_per_tick)
        tr = self.api.trader()
        if isinstance(tr, dict):
            self.trader_id = tr.get("trader_id")
        lim = self.api.limits()
        if isinstance(lim, list) and lim:
            g = sum(float(x.get("gross_limit") or 0) for x in lim) or DEFAULT_LIMIT
            n = sum(float(x.get("net_limit") or 0) for x in lim) or DEFAULT_LIMIT
            self.gross_limit, self.net_limit = int(g), int(n)
        self.cap_gross = int(CAP_FRACTION * self.gross_limit)
        self.cap_net = int(CAP_FRACTION * self.net_limit)
        print(f"[limits] gross {self.gross_limit:,} net {self.net_limit:,} caps {self.cap_gross:,}/{self.cap_net:,}")
        secs = self.api.securities()
        if isinstance(secs, list) and secs:
            self.tickers = [x["ticker"] for x in secs if x.get("ticker")
                            and str(x.get("is_tradeable", True)).lower() != "false"]
        if not self.tickers:
            self.tickers = list(DEFAULT_TICKERS)
        for t in self.tickers:
            self.secs[t] = Sec(t)
        self._apply_securities(secs if isinstance(secs, list) else [], first=True)
        for t, s in self.secs.items():
            print(f"[sec] {t:<4} tick {s.tick} taker fee {s.fee_take:+.4f} maker rebate {s.fee_make:+.4f}")
        self.log = Log(self.tickers)
        print(f"[log] {self.log.tick_path} {self.log.ev_path}", flush=True)

    # ------------------------------------------------------------------------ data intake
    def _apply_securities(self, secs, first=False):
        for row in secs:
            s = self.secs.get(row.get("ticker"))
            if not s:
                continue
            old = s.pos
            s.pos = int(row.get("position") or 0)
            s.vwap = float(row.get("vwap") or 0)
            s.realized = float(row.get("realized") or 0)
            s.unrealized = float(row.get("unrealized") or 0)
            s.last = float(row.get("last") or 0)
            if first:
                qd = row.get("quoted_decimals")
                if qd is not None:
                    s.tick = 10 ** (-int(qd))
                tf, rb = row.get("trading_fee"), row.get("limit_order_rebate")
                if tf is not None and rb is not None:
                    s.fee_take, s.fee_make = float(tf), float(rb)
            elif s.pos != old and self.log:
                d = s.pos - old
                self.log.event(self.now(), self.case_tick, "fill", s.t, "BUY" if d > 0 else "SELL",
                               abs(d), s.mid, s.pos, f"mode={s.mode}", echo=abs(d) >= 1000 or s.mode != "mm")

    def _apply_book(self, s, book):
        if not isinstance(book, dict) or book.get("_error"):
            return
        mine = {oid for oid, r in self.live.items() if r["ticker"] == s.t}

        def best(levels):
            for lv in levels or []:
```

```
            if lv.get("order_id") in mine:
                continue
            if self.trader_id and lv.get("trader_id") == self.trader_id:
                continue
            rem = float(lv.get("quantity", 0)) - float(lv.get("quantity_filled", 0))
            if rem > 0:
                return float(lv["price"])
        return 0.0

    s.bid = best(book.get("bids"))
    s.ask = best(book.get("asks"))

def refresh(self):
    now = self.now()
    futs = {}
    if now - self.last_slow >= SLOW_POLL_SEC:
        futs["case"] = self.pool.submit(self.api.case)
        futs["trader"] = self.pool.submit(self.api.trader)
        self.last_slow = now
    futs["orders"] = self.pool.submit(self.api.open_orders)
    futs["secs"] = self.pool.submit(self.api.securities)
    books = {t: self.pool.submit(self.api.book, t) for t in self.tickers}

    t_snap = time.time()
    orders = futs["orders"].result()
    if isinstance(orders, list):
        self._sync_orders(orders, t_snap)
    secs = futs["secs"].result()
    if isinstance(secs, list):
        self._apply_securities(secs)
    for t, f in books.items():
        self._apply_book(self.secs[t], f.result())
    if "case" in futs:
        c = futs["case"].result()
        if isinstance(c, dict) and not c.get("_error"):
            self.status = c.get("status", "?")
            tick = int(c.get("tick") or 0)
            self.tpp = int(c.get("ticks_per_period") or self.tpp)
            per = c.get("period")
            if per != self.period:
                self.period = per
                self._tick_anchor = None
            if tick > 0:
                if self._tick_anchor is None or tick < self._tick_anchor[1]:
                    self._tick_anchor = (time.time(), tick)
                elif tick - self._tick_anchor[1] >= 3:
                    measured = (time.time() - self._tick_anchor[0]) / (tick - self._tick_anchor[1])
                    if not self.minutes or tick - self._tick_anchor[1] >= 10:
                        self.sec_per_tick = measured
                        if self.time_scale is None or abs(apply_time_scale(measured) - self.time_scale) > 0.1:
                            self.time_scale = apply_time_scale(measured)
            self.case_tick = tick
        tr = futs["trader"].result()
        if isinstance(tr, dict) and tr.get("nlv") is not None:
            self.nlv = float(tr["nlv"])
            if self.nlv0 is None:
                self.nlv0 = self.nlv
    now = self.now()
    for s in self.secs.values():
        before = s.regime
        s.update_signals(now)
        s.update_slow(self.case_tick if self.status == "ACTIVE" else None)
        if s.regime != before and self.log and before != "warmup":
            self.log.event(now, self.case_tick, "regime", s.t,
                           note=f"{before} -> {s.regime} (er={s.er:.2f}, net {s.net_ticks:+.0f}t)",
                           echo=s.regime == "TREND" or before == "TREND")
    if self.nlv is not None and self.nlv0 is not None:
        self.pnl = self.nlv - self.nlv0
    else:
        self.pnl = sum(s.realized + s.unrealized for s in self.secs.values())
    self.peak = max(self.peak, self.pnl)

def _sync_orders(self, snapshot, t_snap):
    seen = set()
    for o in snapshot:
        oid = o.get("order_id")
        if oid is None:
            continue
        seen.add(oid)
        rec = self.live.get(oid)
        if rec is None:
            rec = {"order_id": oid, "ticker": o.get("ticker"), "action": o.get("action"),
                   "price": float(o.get("price") or 0), "t_sent": t_snap, "kind": "quote"}
            self.live[oid] = rec
        rec["quantity"] = int(o.get("quantity") or 0)
        rec["quantity_filled"] = int(o.get("quantity_filled") or 0)
        if not rec.get("seen") and rec.get("state") == "new":
            lat = t_snap - rec["t_sent"]
            if 0 <= lat < 3:
```

```
                self.lat_est += 0.2 * (lat - self.lat_est)
            rec["seen"] = True
            rec["state"] = "open"
        for oid, rec in list(self.live.items()):
            if oid in seen:
                continue
            if rec["state"] == "new" and t_snap - rec["t_sent"] < VISIBILITY_GRACE:
                continue
            if rec["state"] != "gone":
                rec["state"] = "gone"
                rec["t_gone"] = t_snap
            elif t_snap - rec["t_gone"] > GONE_HOLD:
                del self.live[oid]
        for s in self.secs.values():
            s.orders = [r for r in self.live.values() if r["ticker"] == s.t and r["state"] in ("new", "open")]

    def sec_left(self):
        return max(0.0, (self.tpp - self.case_tick) * (self.sec_per_tick or 1.0))

    def unwind_windows(self):
        total = self.tpp * (self.sec_per_tick or 1.0)
        return (min(UNWIND_START_SEC, max(6.0, 0.05 * total)),
                min(UNWIND_HARD_SEC, max(3.0, 0.02 * total)))

    # ------------------------------------------------------------------------ exposure
    def _resting(self, s, side):
        return sum(int(r["quantity"]) for r in self.live.values() if r["ticker"] == s.t and r["action"] == side)

    def _inflight_agg(self, s, side):
        t_wall = time.time()
        return sum(int(r["quantity"]) for r in self.live.values()
                   if r["ticker"] == s.t and r["action"] == side and r.get("kind") == "agg"
                   and t_wall - r["t_sent"] < AGG_INTERVAL)

    def exposure_ok(self, t, side, qty, pending):
        gross = nl = ns = 0
        L_t = S_t = 0
        for name, s in self.secs.items():
            rb = self._resting(s, "BUY") + pending.get((name, "BUY"), 0)
            rs = self._resting(s, "SELL") + pending.get((name, "SELL"), 0)
            L, S = s.pos + rb, s.pos - rs
            if name == t:
                if side == "BUY":
                    L += qty
                else:
                    S -= qty
                L_t, S_t = L, S
            gross += max(abs(L), abs(S)); nl += L; ns += S
        if side == "BUY":
            if nl > self.cap_net:
                return False
            return gross <= self.cap_gross or L_t <= 0
        if ns < -self.cap_net:
            return False
        return gross <= self.cap_gross or S_t >= 0

    def fit_qty(self, t, side, qty, pending):
        qty = int(min(qty, MAX_ORDER))
        while qty >= 1:
            if self.exposure_ok(t, side, qty, pending):
                return qty
            if qty <= 100:
                return 0
            qty = max(100, int(qty * 0.6) // 100 * 100)
        return 0

    def dir_cap(self, s):
        others = sum(abs(x.pos) for x in self.secs.values() if x is not s)
        room = int(DIR_FRACTION * self.cap_gross) - others
        return max(0, min(room, int(DIR_FRACTION * self.cap_net)))

    # ------------------------------------------------------------------------ decisions
    def agg_price(self, s, side):
        ref = s.fast or s.mid
        dev = max(AGG_MAX_DEV * ref, 10 * s.tick)
        if side == "BUY":
            if s.ask <= 0 or s.ask > ref + dev:
                return None
            run = max(0.0, s.vel) * self.lat_est * 1.5
            return s.rnd(min(s.ask + (AGG_THROUGH_TICKS + math.ceil(run)) * s.tick, ref + dev))
        if s.bid <= 0 or s.bid < ref - dev:
            return None
        run = max(0.0, -s.vel) * self.lat_est * 1.5
        return s.rnd(max(s.bid - (AGG_THROUGH_TICKS + math.ceil(run)) * s.tick, ref - dev))

    def decide(self, s, now, sec_left):
        passive, aggressive = [], []
        if s.fast is None or (s.bid <= 0 and s.ask <= 0):
            return passive, aggressive
```

```
        u_start, u_hard = self.unwind_windows()
        unwinding = sec_left <= u_start
        hard = sec_left <= u_hard or self.flattening or now < self.paused_until

        if hard:
            if s.pos != 0:
                side = "SELL" if s.pos > 0 else "BUY"
                px = self.agg_price(s, side)
                if px is not None:
                    aggressive.append((side, px, min(abs(s.pos), AGG_CHUNK)))
            s.mode = "mm"
            return passive, aggressive

        thr = s.dir_threshold()
        tick = s.tick

        # trend engine
        if not unwinding:
            if s.mode == "dir":
                m = s.mid
                if s.dir_side * (m - s.dir_best) > 0:
                    s.dir_best, s.dir_best_t = m, now
                drawdown = s.dir_side * (s.dir_best - m) / tick
                if s.dir_kind == "slow" and s.dir_side * s.gap > thr and s.er >= ER_TREND:
                    s.dir_kind = "fast"
                if s.dir_kind == "slow":
                    trail = max(DIR_TRAIL_TICKS, 3.0 * s.sd_slow * math.sqrt(5))
                    fading = s.dir_side * s.t_slow < SLOW_T_EXIT
                    stall_sec = SLOW_STALL_TICKS * (self.sec_per_tick or 1.0)
                else:
                    trail = max(DIR_TRAIL_TICKS, DIR_TRAIL_VOL_K * s.vol)
                    fading = s.dir_side * s.gap < DIR_EXIT_FRAC * thr and s.dir_side * s.t_slow < SLOW_T_ENTER
                    stall_sec = DIR_STALL_SEC
                stalled = now - s.dir_best_t > stall_sec
                if drawdown > trail or fading or stalled:
                    why = ("trail" if drawdown > trail else "faded" if fading else "stalled")
                    self.log.event(now, self.case_tick, "dir_exit", s.t, "", s.pos, s.mid, s.pos, why, echo=True)
                    s.mode = "cut"
                    s.cool_until = now + DIR_COOLDOWN
                    s.last_exit = (s.dir_side, s.dir_best)
                    self.learn_ride(s, now)
                elif s.dir_kind == "fast":
                    strength = min(1.0, 0.6 + 0.4 * (abs(s.gap) / thr - 1.0))
                    s.dir_target = s.dir_side * max(abs(s.dir_target), int(self.dir_cap(s) * strength))
            elif (now >= s.cool_until and now >= s.dir_off_until and abs(s.gap) > thr
                  and s.er >= ER_TREND and s.net_ticks * s.gap > 0
                  and s.spread_ticks <= DIR_MAX_SPREAD_TICKS and self.dir_cap(s) >= DIR_MIN_SIZE):
                side = 1 if s.gap > 0 else -1
                fresh = (s.last_exit is None or s.last_exit[0] != side
                         or side * (s.mid - s.last_exit[1]) > 0.5 * thr * tick)
                if not fresh:
                    s.trend_since = None
                elif s.trend_since is None or s.trend_since[1] != side:
                    s.trend_since = (now, side)
                elif now - s.trend_since[0] >= DIR_CONFIRM_SEC:
                    s.mode, s.dir_side, s.dir_best, s.dir_best_t = "dir", side, s.mid, now
                    s.dir_entry_mid = s.mid
                    s.dir_kind = "fast"
                    s.dir_target = side * int(self.dir_cap(s) * 0.6)
                    self.log.event(now, self.case_tick, "dir_enter", s.t, "LONG" if side > 0 else "SHORT",
                                   abs(s.dir_target), s.mid, s.pos,
                                   f"gap={s.gap:.1f}t thr={thr:.1f}t er={s.er:.2f}", echo=True)
            elif (now >= s.cool_until and now >= s.dir_off_until
                  and abs(s.t_slow) >= SLOW_T_ENTER * s.dir_mult and abs(s.net_slow) >= SLOW_MIN_TICKS
                  and s.spread_ticks <= DIR_MAX_SPREAD_TICKS and self.dir_cap(s) >= DIR_MIN_SIZE
                  and (s.last_exit is None or s.last_exit[0] != (1 if s.t_slow > 0 else -1)
                       or (1 if s.t_slow > 0 else -1) * (s.mid - s.last_exit[1]) > 0)):
                side = 1 if s.t_slow > 0 else -1
                s.mode, s.dir_side, s.dir_best, s.dir_best_t = "dir", side, s.mid, now
                s.dir_entry_mid = s.mid
                s.dir_kind = "slow"
                s.dir_target = side * int(self.dir_cap(s) * SLOW_FRACTION)
                self.log.event(now, self.case_tick, "dir_enter", s.t, "LONG" if side > 0 else "SHORT",
                               abs(s.dir_target), s.mid, s.pos, f"slow t={s.t_slow:.1f}", echo=True)
            else:
                s.trend_since = None
        elif s.mode == "dir":
            s.mode = "cut"

        if s.mode == "dir":
            diff = s.dir_target - s.pos - (self._resting(s, "BUY") if s.dir_side > 0 else -self._resting(s, "SELL"))
            if s.dir_side * diff >= 200:
                side = "BUY" if s.dir_side > 0 else "SELL"
                px = self.agg_price(s, side)
                if px is not None:
                    aggressive.append((side, px, min(abs(diff), AGG_CHUNK)))
            return passive, aggressive
```

```
        if s.mode == "cut":
            if s.pos == 0:
                s.mode = "mm"
            else:
                side = "SELL" if s.pos > 0 else "BUY"
                px = self.agg_price(s, side)
                if px is not None:
                    aggressive.append((side, px, min(abs(s.pos), AGG_CHUNK)))
                return passive, aggressive

        # market-making loss pause
        if s.pnl_hist and now >= s.mm_off_until:
            peak = max(v for _, v in s.pnl_hist)
            speed = min(4.0, max(1.0, 9.0 / self.sec_per_tick)) if self.sec_per_tick else 1.0
            if peak - s.est_pnl > MM_LOSS_LIMIT * speed:
                pause = MM_PAUSE_SEC[min(s.mm_pauses, len(MM_PAUSE_SEC) - 1)]
                s.mm_pauses += 1
                s.mm_off_until = now + pause
                s.size_mult = max(0.25, s.size_mult * 0.5)
                s.size_t = now
                s.pnl_hist.clear()
                self.log.event(now, self.case_tick, "regime", s.t, note=f"mm pause {pause:.0f}s", echo=True)
        mm_paused = now < s.mm_off_until

        # market-making inventory stop
        if s.pos != 0:
            per_share = s.unrealized / abs(s.pos) if s.unrealized else (
                (s.mid - s.vwap) * (1 if s.pos > 0 else -1) if s.vwap else 0.0)
            if abs(per_share) > 0.5 * max(s.mid, 1):
                per_share = (s.mid - s.vwap) * (1 if s.pos > 0 else -1) if s.vwap else 0.0
            loss_ticks = -per_share / tick
            stop = max(STOP_MIN_TICKS, STOP_VOL_K * s.vol)
            against_trend = (s.pos > 0 and s.gap < -0.7 * thr) or (s.pos < 0 and s.gap > 0.7 * thr)
            if loss_ticks > stop or against_trend:
                side = "SELL" if s.pos > 0 else "BUY"
                px = self.agg_price(s, side)
                if px is not None:
                    aggressive.append((side, px, min(abs(s.pos), AGG_CHUNK)))
                    if s.pos > 0:
                        s.no_buy_until = now + STOP_COOLDOWN
                    else:
                        s.no_sell_until = now + STOP_COOLDOWN
                    if self._inflight_agg(s, side) == 0:
                        s.stop_times.append(now)
                    while s.stop_times and now - s.stop_times[0] > TOXIC_WINDOW:
                        s.stop_times.popleft()
                    if len(s.stop_times) >= TOXIC_STOPS and now >= s.toxic_until:
                        s.toxic_until = now + TOXIC_SEC
                        s.stop_times.clear()
                    if self._inflight_agg(s, side) == 0:
                        self.log.event(now, self.case_tick, "stop", s.t, side, min(abs(s.pos), AGG_CHUNK), px,
                                       s.pos, f"loss={loss_ticks:.1f}t stop={stop:.1f}t", echo=True)
                    return passive, aggressive

        # market-making quotes
        bb, ba = s.bid, s.ask
        fair = s.fair or s.mid
        inv = s.pos / MM_INV_CAP
        r = min(1.0, abs(inv))
        toxic = now < s.toxic_until
        stressed = toxic or s.vstate == "stressed"
        calm = s.vstate == "calm" and not toxic
        if bb <= 0 or ba <= 0 or s.spread_ticks >= WIDE_SPREAD_TICKS:
            edge = max(WIDE_EDGE_TICKS, 2 * s.vol) * tick
            bid, ask = fair - edge, fair + edge
            if bb > 0:
                bid = max(bid, bb + tick)
            if ba > 0:
                ask = min(ask, ba - tick)
            size = WIDE_SIZE
        else:
            if s.spread_ticks >= IMPROVE_MIN_SPREAD and not stressed:
                bid, ask = bb + tick, ba - tick
                if ask - bid < tick:
                    if inv > 0:
                        bid = bb
                    else:
                        ask = ba
            else:
                bid, ask = bb, ba
            if stressed:
                bid -= tick
                ask += tick
            healthy_wide = WIDE_OK_MIN <= s.spread_ticks <= WIDE_OK_MAX and s.regime != "TREND"
            size = QUOTE_SIZE_CALM if (calm or healthy_wide or self.widest == s.t) else QUOTE_SIZE
        if stressed:
            size //= 2
        if s.size_mult < 1.0 and now - s.size_t > 60.0:
            s.size_mult, s.size_t = min(1.0, s.size_mult * 1.25), now
```

```
        size = int(size * s.size_mult)
        if self.pnl < -50000:
            size //= 4
        elif self.pnl < -25000:
            size //= 2
        fade = int(round(FADE_MAX_TICKS * r ** 1.5))
        if inv > 0:
            bid -= fade * tick
            if r > 0.4:
                ask -= tick
        elif inv < 0:
            ask += fade * tick
            if r > 0.4:
                bid += tick
        run = s.vel * self.lat_est
        if run > 0.5:
            ask += math.ceil(run) * tick
        elif run < -0.5:
            bid -= math.ceil(-run) * tick
        if ba > 0:
            bid = min(bid, ba - tick)
        if bb > 0:
            ask = max(ask, bb + tick)
        if ask - bid < tick:
            return passive, aggressive
        add_q = 0.0 if r >= HALT_FRAC else size * math.exp(-EXP_K * r)
        red_q = min(MAX_ORDER, size * (1.0 + r))
        if inv > 0:
            bid_q, ask_q = add_q, red_q
        elif inv < 0:
            bid_q, ask_q = red_q, add_q
        else:
            bid_q = ask_q = size
        lean = s.gap / thr if thr else 0.0
        if (lean > 0.5 or s.t_slow >= SLOW_T_LEAN) and 0 <= s.pos < LEAN_MAX_INV:
            ask_q = 0
        if (lean < -0.5 or s.t_slow <= -SLOW_T_LEAN) and -LEAN_MAX_INV < s.pos <= 0:
            bid_q = 0
        if s.t_slow <= -SLOW_T_LEAN and s.pos > 0:
            bid_q = 0
        if s.t_slow >= SLOW_T_LEAN and s.pos < 0:
            ask_q = 0
        if run >= SPEED_PULL_TICKS:
            ask_q = min(ask_q, max(0, s.pos))
        if run <= -SPEED_PULL_TICKS:
            bid_q = min(bid_q, max(0, -s.pos))
        if unwinding or mm_paused or self.over > 0:
            bid_q = min(bid_q, max(0, -s.pos))
            ask_q = min(ask_q, max(0, s.pos))
        if now < s.no_buy_until:
            bid_q = min(bid_q, max(0, -s.pos))
        if now < s.no_sell_until:
            ask_q = min(ask_q, max(0, s.pos))
        if bid_q >= 100:
            passive.append(("BUY", s.rnd(bid), int(bid_q) // 100 * 100))
        if ask_q >= 100:
            passive.append(("SELL", s.rnd(ask), int(ask_q) // 100 * 100))
        return passive, aggressive

    def learn_ride(self, s, now):
        ride = s.dir_side * (s.mid - s.dir_entry_mid) / s.tick
        s.pnl_hist.clear()
        if ride <= 0:
            s.rides[1] += 1
            s.lose_streak += 1
            s.dir_mult = min(DIR_MULT_MAX, s.dir_mult * 1.4)
            if s.lose_streak >= CHOP_LOCKOUT_RIDES:
                s.dir_off_until = now + CHOP_LOCKOUT_SEC
                s.lose_streak = 0
        else:
            s.rides[0] += 1
            s.lose_streak = 0
            s.dir_mult = max(1.0, s.dir_mult * 0.8)
        self.log.event(now, self.case_tick, "regime", s.t, note=f"ride {ride:+.0f}t entry bar x{s.dir_mult:.2f}")

    # ------------------------------------------------------------------------ execution
    def execute(self, plans):
        now = self.now()
        cancels, places = [], []
        pending = {}
        for t, (passive, aggressive) in plans.items():
            s = self.secs[t]
            want = {side: (px, q) for side, px, q in passive}
            agg_sides = {side for side, _, _ in aggressive}
            t_wall = time.time()
            for o in s.orders:
                side = o["action"]
                if o.get("t_cancel"):
                    if t_wall - o["t_cancel"] > CANCEL_RESEND and o["state"] == "open":
```

```
                    cancels.append((s, o))
                continue
            if o.get("kind") == "agg":
                if t_wall - o["t_sent"] > AGG_TTL and o["state"] == "open":
                    cancels.append((s, o))
                continue
            rem = int(o["quantity"]) - int(o.get("quantity_filled") or 0)
            w = want.get(side)
            keep = (w is not None and side not in agg_sides
                    and abs(float(o["price"]) - w[0]) < s.tick / 2 and rem >= KEEP_FRAC * w[1])
            if keep:
                want[side] = None
            elif o["state"] == "open":
                cancels.append((s, o))
            elif w is not None and abs(float(o["price"]) - w[0]) < s.tick / 2:
                want[side] = None
        for side, v in want.items():
            if v is not None:
                places.append((s, side, v[0], v[1], "quote"))
        for side, px, q in aggressive:
            q = int(q) - self._inflight_agg(s, side)
            if q >= 100 or (q > 0 and q >= abs(s.pos) - self._inflight_agg(s, side) and s.pos != 0):
                places.append((s, side, px, q, "agg"))

    if cancels:
        futs = [self.pool.submit(self.api.cancel, o["order_id"]) for _, o in cancels]
        for (s, o), f in zip(cancels, futs):
            f.result()
            o["t_cancel"] = time.time()
            self.cancels += 1

    places.sort(key=lambda x: 0 if x[4] == "agg" else (1 if x[0].t == self.widest else 2))
    jobs = []
    for s, side, px, q, kind in places:
        reducing = kind == "agg" and ((side == "BUY" and s.pos < 0) or (side == "SELL" and s.pos > 0))
        if reducing:
            q = min(int(q), abs(s.pos))
            net_now = sum(x.pos for x in self.secs.values())
            infl = sum(self._inflight_agg(x, side) for x in self.secs.values())
            if side == "BUY" and net_now + infl + q > self.cap_net:
                q = max(0, self.cap_net - net_now - infl)
            if side == "SELL" and net_now - infl - q < -self.cap_net:
                q = max(0, net_now - infl + self.cap_net)
        else:
            q = self.fit_qty(s.t, side, q, pending)
        if q < 1:
            continue
        pending[(s.t, side)] = pending.get((s.t, side), 0) + q
        jobs.append((s, side, px, q, kind, time.time(), self.pool.submit(self.api.place, s.t, side, q, px)))
    for s, side, px, q, kind, t_sent, f in jobs:
        r = f.result()
        self.places += 1
        ok = isinstance(r, dict) and not r.get("_error")
        if ok and r.get("order_id") is not None:
            self.live[r["order_id"]] = {
                "order_id": r["order_id"], "ticker": s.t, "action": side, "price": px,
                "quantity": q, "quantity_filled": int(r.get("quantity_filled") or 0),
                "t_sent": t_sent, "kind": kind, "state": "new", "seen": False}
        self.log.event(now, self.case_tick, "place", s.t, side, q, px, s.pos,
                       kind if ok else f"{kind} REJECTED {r}", echo=(kind == "agg" or not ok))

# ---------------------------------------------------------------------- main loop
def heartbeat(self, now, sl):
    if now - self.last_beat < HEARTBEAT_SEC:
        return
    self.last_beat = now
    gross = sum(abs(s.pos) for s in self.secs.values())
    net = sum(s.pos for s in self.secs.values())
    parts = [f"{s.t} {s.pos:+6d} {s.bid:.2f}/{s.ask:.2f} {s.mode} [{s.regime}/{s.vstate} er{s.er:.2f}]"
             for s in self.secs.values()]
    print(f"[{now:7.1f}s] tick {self.case_tick}/{self.tpp} ~{sl:5.0f}s left | P&L ${self.pnl:+,.0f} | "
          f"gross {gross:,} net {net:+,} | " + " | ".join(parts), flush=True)

def write_tick(self, now, sl):
    gross = sum(abs(s.pos) for s in self.secs.values())
    net = sum(s.pos for s in self.secs.values())
    row = [f"{now:.3f}", self.case_tick, f"{sl:.1f}", f"{self.pnl:.2f}", f"{self.nlv or 0:.2f}", gross, net]
    for t in self.tickers:
        s = self.secs[t]
        row += [s.bid, s.ask, f"{(s.fair or 0):.4f}", f"{s.gap:.2f}", f"{s.vol:.2f}", f"{s.er:.2f}", s.regime,
                f"{s.bb_z:.2f}", s.vstate, f"{s.t_slow:.2f}", s.pos, s.mode, f"{s.realized + s.unrealized:.2f}"]
    self.log.tick_row(row)

def run(self):
    self.setup()
    waiting_printed = 0.0
    was_active = False
    while self.running:
        t_loop = time.time()
```

```
            try:
                self.refresh()
            except Exception as e:
                print(f"[warn] refresh error: {e!r}", flush=True)
                time.sleep(0.2)
                continue
            now = self.now()
            if self.status != "ACTIVE":
                if was_active:
                    print(f"[case] over. Final P&L ${self.pnl:+,.2f}", flush=True)
                    was_active = False
                if now - waiting_printed > 3:
                    print(f"[wait] case status={self.status} - waiting for ACTIVE...", flush=True)
                    waiting_printed = now
                time.sleep(0.2)
                continue
            if not was_active:
                print(f"[case] ACTIVE at tick {self.case_tick} - trading.", flush=True)
                was_active = True
            sl = self.sec_left()
            if not self.sec_per_tick:
                sl = 1e9
            if self.pnl < self.peak - BREAKER_DD and now >= self.paused_until:
                self.paused_until = now + BREAKER_PAUSE
                self.log.event(now, self.case_tick, "breaker", note=f"pnl {self.pnl:,.0f} peak {self.peak:,.0f}",
                               echo=True)
                self.peak = self.pnl
            cands = [x for x in self.secs.values()
                     if WIDE_OK_MIN <= x.spread_ticks <= WIDE_OK_MAX and x.regime != "TREND"]
            self.widest = max(cands, key=lambda x: x.spread_ticks).t if cands else None
            gross = sum(abs(x.pos) for x in self.secs.values())
            net = abs(sum(x.pos for x in self.secs.values()))
            self.over = max(gross - self.cap_gross, net - self.cap_net, 0)
            hard_line = int(SAFETY_FRACTION * min(self.gross_limit, self.net_limit))
            plans = {}
            for t in self.tickers:
                try:
                    plans[t] = self.decide(self.secs[t], now, sl)
                except Exception as e:
                    print(f"[warn] decide {t}: {e!r}", flush=True)
                    plans[t] = ([], [])
            if max(gross, net) > hard_line:
                big = max(self.secs.values(), key=lambda x: abs(x.pos))
                side = "SELL" if big.pos > 0 else "BUY"
                px = self.agg_price(big, side) or (big.bid - 3 * big.tick if side == "SELL" else big.ask + 3 * big.tick)
                cut = min(AGG_CHUNK, max(gross, net) - self.cap_gross + 500, abs(big.pos))
                if cut > 0 and px > 0:
                    pas, agg = plans.get(big.t, ([], []))
                    plans[big.t] = ([p for p in pas if p[0] == side], agg + [(side, px, cut)])
                    self.log.event(now, self.case_tick, "safety", big.t, side, cut, px, big.pos,
                                   f"gross {gross:,} net {net:,} above {hard_line:,}", echo=True)
            try:
                self.execute(plans)
            except Exception as e:
                print(f"[warn] execute error: {e!r}", flush=True)
            self.write_tick(now, sl)
            self.loops += 1
            self.heartbeat(now, sl)
            el = time.time() - t_loop
            if el < LOOP_MIN_DT:
                time.sleep(LOOP_MIN_DT - el)

    def shutdown(self):
        print("\n[stop] cancelling all orders and flattening", flush=True)
        try:
            self.api.cancel_all()
            self.flattening = True
            t_end = time.time() + 6
            while time.time() < t_end:
                self.refresh()
                if all(s.pos == 0 for s in self.secs.values()) or self.status != "ACTIVE":
                    break
                self.execute({t: self.decide(self.secs[t], self.now(), 0) for t in self.tickers})
                time.sleep(0.1)
            self.api.cancel_all()
            self.refresh()
        except Exception as e:
            print(f"[stop] {e!r}")
        pos = {t: s.pos for t, s in self.secs.items()}
        print(f"[stop] final positions {pos} P&L ${self.pnl:+,.2f}", flush=True)
        if self.log:
            self.log.event(self.now(), self.case_tick, "end", note=f"pnl {self.pnl:.2f} pos {pos}")
            self.log.close()


def main():
    ap = argparse.ArgumentParser(description="RIT ALGO2e strategy")
    ap.add_argument("key", help="RIT API key")
    ap.add_argument("port", type=int, help="RIT API port")
```

```
    ap.add_argument("minutes", nargs="?", type=float, default=None, help="case length in minutes (optional)")
    ap.add_argument("--host", default="localhost")
    args = ap.parse_args()

    bot = Bot(RIT(args.key, f"http://{args.host}:{args.port}/v1"), minutes=args.minutes)
    signal.signal(signal.SIGINT, lambda sig, frm: setattr(bot, "running", False))
    try:
        bot.run()
    finally:
        bot.shutdown()


if __name__ == "__main__":
    main()
```